Product-company data interview preparation — start here

Who this programme is for

A fresher learning independently for entry-level Data Analyst, Product Analyst, Business Intelligence Analyst or Junior Data Engineer roles. It assumes basic computer literacy but does **not** assume interview-level SQL, DBMS, Python or analytics knowledge.

This is a course, workbook and assessment system. It is deliberately organised as a learning sequence: concept → worked example → guided exercise → independent exercise → solution → concept test → cumulative mock. Do not jump straight to answer keys.

What this programme is—and is not

It teaches transferable foundations used across product companies. It is not a promise that every company uses the same interview rounds, and it is not a list of leaked questions. Job descriptions differ: an analyst role may emphasise SQL, product metrics, statistics and BI; an engineering role may emphasise SQL, Python, DBMS, pipelines and cloud. Some companies also test DSA, aptitude, take-home work or tool-specific skills.

Learning rules

1. **Attempt first.** Keep solutions in a separate file or print the exercise pages without answer sections.

2. **Write, don't just read.** Type every SQL query and Python solution. Change sample data and rerun.

3. **Explain aloud.** For every solution, explain the grain, assumptions, edge cases and complexity.

4. **Maintain an error log.** Record the error, why it happened, the corrected rule and a new example.

5. **Use spaced repetition.** Reattempt missed questions after 1 day, 3 days and 7 days.

6. **No memorised claims.** For projects and resumes, explain only what was actually implemented.

7. **Use PostgreSQL for SQL practice.** Syntax differs across PostgreSQL, MySQL, SQL Server, Oracle and SQLite. Learn the concept, then check the target dialect.

Recommended 20-week route

| Weeks | Study block | Required output |

|---|---|---|

| 1 | Relational data, tables, keys, constraints and SQL setup | Create a small schema and load sample rows |

| 2–3 | SQL querying, joins, aggregation and NULLs | 35 beginner queries with verified results |

| 4 | Subqueries, CTEs, set operations and conditional logic | 15 intermediate queries |

| 5–6 | Window functions, dates, deduplication and advanced SQL | 20 interview queries under time |

| 7 | DBMS internals, normalisation, transactions and ACID | Explain and demonstrate transactions |

| 8 | Isolation, locks, indexes and query plans | Investigate 3 deliberately slow queries |

| 9–10 | Python fundamentals and problem solving | 25 small programs without pandas |

| 11 | Files, exceptions, modules, OOP and testing | Build a validated CSV processing script |

| 12 | pandas and data cleaning | Reproduce an analysis from raw CSV |

| 13 | Statistics and experimentation | Explain a product experiment and uncertainty |

| 14 | Excel, Power Query and Power BI | Build and reconcile a small dashboard |

| 15 | Data modelling and warehouse fundamentals | Design a star schema and explain its grain |

| 16–17 | Data engineering pipelines, APIs, reliability and cloud concepts | Build a rerunnable batch pipeline |

| 18 | Project deep-dives and resume defence | Explain each project in 5 minutes |

| 19 | Analyst and engineering role-specific practice | Complete one role-specific mock each |

| 20 | Full interview simulations and revision | Complete two timed mock loops |

A student with less time can compress the calendar, but should not skip SQL, DBMS, Python, project explanation or timed practice.

Daily routine (2–3 hours)

• 15 min: recall yesterday's material without notes.

• 35 min: learn one new concept.

• 45 min: solve guided exercises.

• 35 min: solve independent exercises.

• 20 min: explain the concept aloud and write an error-log entry.

• 10 min: review a previously missed question.

How to use the solutions

A solution is not just code. Compare:

• Does it answer the exact question?

• Does it preserve the intended row grain?

• What happens with NULL, duplicates, ties, empty input or invalid values?

• Is there a simpler or more scalable approach?

• Can you explain why it works?

Readiness gates

SQL: solve joins, aggregation, subqueries, windows, date logic and deduplication accurately under time.

DBMS: explain keys, normalisation, ACID, isolation, deadlocks, indexes and execution plans with examples.

Python: solve list/dictionary/set problems, validate input, test edge cases and explain complexity.

Analytics: define metrics, interpret distributions and uncertainty, diagnose product changes.

BI: model data, write measures, reconcile totals and design a decision-oriented dashboard.

Data engineering: design an incremental, idempotent, observable pipeline with failure recovery.

Projects: trace data from source to output and explain limitations honestly.

Tool setup

Recommended local setup:

• PostgreSQL + pgAdmin or DBeaver for SQL/DBMS exercises.

• Python 3.11+ and VS Code.

• Jupyter Notebook for pandas/statistics exercises.

• Git and GitHub for version control.

• Power BI Desktop for BI exercises (Windows; on macOS use a suitable alternative such as Power BI Service where available, or Tableau Public/Looker Studio for visualisation practice).

Use the smallest tools needed. Do not spend weeks configuring infrastructure instead of learning the fundamentals.

External practice — how to use the links in each module

Use the external platforms listed at the end of each module, not a random playlist of sites. The internal exercises teach the concept; external platforms provide independent problems and automated feedback. Keep a tracker with columns: concept, platform, problem URL/title, first attempt date, result, mistake, retry date.

Rules: do not copy solutions on the first attempt; solve at least one problem without a timer, then repeat under a timer; write down the dialect/language used; re-solve every failed problem after 1, 3 and 7 days. Free access and platform content can change; check the platform before planning a paid subscription.

Module 1 — Relational data and SQL foundations

Learning outcomes

By the end, you should be able to describe a relational table, identify its grain, define keys and constraints, write simple SELECT queries, filter and sort rows, and explain NULL.

1. Tables, rows, columns and grain

A table stores records with a consistent set of attributes. A row is one record. The **grain** states what one row represents. Example: in `order_items`, one row may represent one product line within one order—not one whole order.

Always identify grain before joining or aggregating. If an order has three item rows, joining the order to items produces three rows for that order.

2. Keys

• Primary key: chosen unique, non-null identifier.

• Candidate key: minimal set of columns that can uniquely identify a row.

• Foreign key: references a key in another table.

• Composite key: uses multiple columns.

• Natural key: business meaning (email, product code); surrogate key: generated identifier.

A key is not merely a column called `id`; uniqueness must be true in the data.

3. Constraints

`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`, `PRIMARY KEY`, `FOREIGN KEY`.

Constraints protect data even when multiple applications write to the database. Application validation alone is not enough.

4. Basic query shape

```
SELECT customer_id, city
FROM customers
WHERE city = 'Mysuru'
ORDER BY customer_id
LIMIT 10;
```

• SELECT chooses output columns/expressions.

• FROM identifies source.

• WHERE filters rows.

• ORDER BY sorts.

• LIMIT restricts returned rows (syntax differs by DBMS).

5. Operators and filtering

Comparison: =, <>, >, >=, <, <=.

Logical: `AND`, `OR`, `NOT`.

Range: `BETWEEN` (inclusive endpoints).

Set membership: `IN`.

Pattern: `LIKE` (% any sequence, _ one character).

NULL: `IS NULL`, `IS NOT NULL`.

6. NULL is not zero or empty text

NULL means missing/unknown/not applicable. `NULL = NULL` is not TRUE; it is UNKNOWN. Use `IS NULL`. Aggregates such as `SUM(column)` ignore NULL values; `COUNT(*)` counts rows and `COUNT(column)` counts non-NULL values.

Worked example

```
SELECT product_id,
       quantity * unit_price AS line_value
FROM order_items
WHERE quantity >= 2
ORDER BY line_value DESC;
```

If either quantity or price is NULL, `line_value` is NULL. Decide whether such rows should be excluded, flagged or treated as invalid; do not silently replace missing prices with zero without a business rule.

Guided exercises

Use:

```
customers(customer_id, customer_name, city, signup_date)
products(product_id, product_name, category, list_price, active)
```

1. Return all customers from Bengaluru.

2. Return products with list price between 500 and 1500.

3. Find inactive products.

4. Return customers whose city is missing.

5. Return the 5 most recently registered customers.

6. Find product names containing the word `phone`.

7. Return distinct cities, including the NULL city if present.

8. Return products where price is missing or non-positive.

Solutions

```
-- 1
SELECT * FROM customers WHERE city = 'Bengaluru';

-- 2
SELECT * FROM products WHERE list_price BETWEEN 500 AND 1500;

-- 3
SELECT * FROM products WHERE active = FALSE;

-- 4
SELECT * FROM customers WHERE city IS NULL;

-- 5
SELECT * FROM customers
ORDER BY signup_date DESC, customer_id DESC
LIMIT 5;

-- 6
SELECT * FROM products WHERE product_name ILIKE '%phone%';

-- 7
SELECT DISTINCT city FROM customers;

-- 8
SELECT * FROM products
WHERE list_price IS NULL OR list_price <= 0;
```

`ILIKE` is PostgreSQL-specific. In other systems use an appropriate case-insensitive comparison.

Independent exercises

9. Find customers who signed up during 2025 using a half-open date range.

10. Return products whose category is either `Electronics` or `Home`.

11. Sort products by category ascending and price descending, with NULL prices last.

12. Find duplicate customer emails in a hypothetical table `customer_contacts(customer_id, email)`.

Solutions

```
-- 9
SELECT * FROM customers
WHERE signup_date >= DATE '2025-01-01'
  AND signup_date <  DATE '2026-01-01';

-- 10
SELECT * FROM products
WHERE category IN ('Electronics', 'Home');

-- 11
SELECT * FROM products
ORDER BY category ASC, list_price DESC NULLS LAST;

-- 12
SELECT email, COUNT(*) AS occurrences
```

```
FROM customer_contacts
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Concept test

1. Can a primary key contain NULL? No.

2. Does `COUNT(column)` count NULL? No.

3. Is NULL the same as zero? No.

4. Why is grain important? It determines what a row means and prevents incorrect joins/aggregates.

5. Is a foreign key automatically NOT NULL? No; add NOT NULL if every row must reference a parent.

Interview challenge

A table has 10,000 rows and 8,000 distinct order IDs. What could explain the difference? Possible answer: the table may be at order-line grain, with multiple rows per order. Do not assume the rows are duplicate transactions.

External practice for this module

Complete these in order, alongside the exercises above:

1. SQLBolt — interactive lessons — complete lessons 1–6 first for SELECT, filtering, sorting and basic joins. Then complete the remaining lessons as revision.

2. HackerRank SQL — Basic Select — solve at least 8 Basic Select problems, including filtering, sorting, pattern matching and NULL handling.

3. SQLZoo — SELECT basics and SELECT from World — solve the tutorial tasks without looking at answers.

4. LeetCode SQL 50 — complete the Select section. Record which questions required a second attempt.

Exit requirement: write 10 SELECT queries from a fresh schema without notes; explain `NULL`, `DISTINCT`, ordering and date boundaries. Use SQLite on SQLBolt/SQLZoo for fundamentals, then repeat representative queries in PostgreSQL.

Module 2 — SQL joins, aggregation and intermediate querying

Learning outcomes

Choose the correct join, predict row multiplication, aggregate at the correct grain, use CASE, subqueries, CTEs and set operations.

1. Join types

• INNER JOIN: only matching rows.

• LEFT JOIN: all left rows; unmatched right columns become NULL.

• RIGHT JOIN: all right rows and matches.

• FULL OUTER JOIN: matched and unmatched rows from both sides.

• CROSS JOIN: every left row paired with every right row.

• Self join: a table joined to itself.

2. The join multiplication problem

Suppose order 17 has four item rows. `orders JOIN order_items` returns four rows for order 17. Therefore `SUM(orders.order_amount)` after this join counts the order amount four times.

Correct approaches:

• aggregate at order grain before joining;

• calculate item-level revenue from item rows;

• use `EXISTS` when only checking whether a related row exists.

3. Aggregation

`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

`GROUP BY` forms groups. `HAVING` filters groups. WHERE filters source rows before grouping.

4. CASE and conditional aggregation

```
SELECT customer_id,
       SUM(CASE WHEN status = 'PAID' THEN amount ELSE 0 END) AS paid_amount,
       SUM(CASE WHEN status = 'REFUNDED' THEN amount ELSE 0 END) AS refunded_amount
FROM payments
GROUP BY customer_id;
```

5. Subqueries and EXISTS

Use a scalar subquery for one value, a correlated subquery for row-dependent logic, and `EXISTS`/`NOT EXISTS` for existence checks. `NOT IN` can behave unexpectedly if the subquery returns NULL.

6. CTEs

A Common Table Expression names a query result for readability and decomposition. It is not guaranteed to be materialised; optimiser behaviour differs by database/version.

7. UNION vs UNION ALL

`UNION` removes duplicates; `UNION ALL` preserves all rows and is usually cheaper. Both require compatible column counts/types.

Worked exercise schema

```
departments(department_id, department_name)
employees(employee_id, employee_name, department_id, salary, manager_id)
```

Guided tasks and solutions

1. List employees with department names.

```
SELECT e.employee_id, e.employee_name, d.department_name
FROM employees e
```

```
LEFT JOIN departments d
  ON d.department_id = e.department_id;
```

2. Count employees per department, including departments with zero employees.

```
SELECT d.department_id, d.department_name,
       COUNT(e.employee_id) AS employee_count
FROM departments d
LEFT JOIN employees e
  ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;
```

Use `COUNT(e.employee_id)`, not `COUNT(*)`, because the unmatched LEFT JOIN row still exists.

3. Find departments with average salary above 80000.

```
SELECT d.department_name, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > 80000;
```

4. Find employees earning more than their department average.

```
SELECT e.*
FROM employees e
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

5. Find employees who manage at least one other employee.

```
SELECT m.employee_id, m.employee_name
FROM employees m
WHERE EXISTS (
  SELECT 1 FROM employees r
  WHERE r.manager_id = m.employee_id
);
```

6. Return employees and their manager names.

```
SELECT e.employee_name AS employee,
       m.employee_name AS manager
FROM employees e
LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

Independent exercises

7. Find the second-highest distinct salary.

8. Find employees whose salary is above the company average.

9. Find employees who do not belong to a valid department.

10. Return department headcount and payroll, including empty departments.

11. Given `web_events(user_id, event_type)`, count unique users who performed `signup` and `purchase` separately.

12. Explain when `UNION ALL` is correct for appending daily source extracts.

Solutions

```
-- 7: handles ties by returning the second distinct value
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 8
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- 9: orphan references (if FK is not enforced)
SELECT e.*
FROM employees e
LEFT JOIN departments d ON d.department_id = e.department_id
WHERE e.department_id IS NOT NULL
  AND d.department_id IS NULL;

-- 10
SELECT d.department_name,
       COUNT(e.employee_id) AS headcount,
       COALESCE(SUM(e.salary), 0) AS payroll
FROM departments d
LEFT JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name;

-- 11
SELECT
  COUNT(DISTINCT user_id) FILTER (WHERE event_type = 'signup') AS signups,
  COUNT(DISTINCT user_id) FILTER (WHERE event_type = 'purchase') AS purchasers
FROM web_events;
```

`FILTER` is PostgreSQL syntax. A portable alternative is `COUNT(DISTINCT CASE WHEN ... THEN user_id END)`.

Common mistakes to avoid

• Joining two one-to-many tables before aggregating.

• Counting rows after a LEFT JOIN rather than counting a non-null right-side key.

• Filtering right-table columns in WHERE when you intended to preserve unmatched left rows.

• Using DISTINCT to hide a wrong join.

• Forgetting that NULL values are ignored by most aggregates.

Mini assessment (20 minutes)

Write queries for: (a) top 3 departments by payroll, (b) employees with no manager, (c) managers with at least 4 direct reports, (d) department salary range, (e) employees whose salary is within 10% of department average. Mark 2 points each for correctness and 2 for explaining assumptions.

External practice for this module

1. HackerRank SQL — Aggregation — solve at least 8 aggregation questions; include GROUP BY, HAVING, COUNT and conditional calculations.

2. LeetCode SQL 50 — complete Basic Joins, Basic Aggregate Functions and Sorting and Grouping sections.

3. DataLemur SQL interview questions — choose 8 questions tagged around joins, aggregations, GROUP BY or business metrics. Attempt before opening hints.

4. PostgreSQL Exercises — complete the joins and aggregates sections against its sample database.

Exit requirement: solve 12 unseen join/aggregation questions. For each, state the grain before writing SQL and explain why a LEFT JOIN does or does not preserve unmatched rows. Include at least two questions where a one-to-many join could inflate a metric.

Module 3 — Advanced SQL for product-company screens

Learning outcomes

Use window functions, date logic, ranking, deduplication, rolling metrics, gap detection and execution plans.

1. Window functions

A window function calculates across related rows without collapsing them like GROUP BY.

• `ROW_NUMBER`: unique sequence.

• `RANK`: ties share rank; gaps follow.

• `DENSE_RANK`: ties share rank; no gaps.

• `LAG`/`LEAD`: previous/next row value.

• Aggregate windows: running sum, moving average, cumulative count.

2. Latest record per entity

Use `ROW_NUMBER() OVER (PARTITION BY key ORDER BY timestamp DESC, stable_tie_breaker DESC)`. Never rely on arbitrary ordering when timestamps tie.

3. Date boundaries

Prefer half-open intervals: `>= start AND < next_period_start`. This avoids missing timestamps later on the final day. Be explicit about timezone and whether a timestamp is stored in UTC.

4. Missing calendar periods

Grouping only returns periods present in the data. For a complete monthly series, join to a calendar/date dimension and fill missing activity with zero where that is semantically valid.

5. Rolling windows

A `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` frame means seven rows, not necessarily seven calendar days. Create a daily date spine before using it as a seven-day metric.

6. Deduplication

Define the business key, version ordering and winner. Preserve raw data. A duplicate can mean repeated delivery of the same event, not two legitimate identical purchases.

7. Query performance

• Inspect `EXPLAIN` and, where safe, `EXPLAIN ANALYZE`.

• Compare estimated and actual row counts.

• Check scan type, join algorithm, sort, aggregate and spills.

• Use selective predicates and project only needed columns.

• Keep indexed filter columns sargable.

• Indexes have write/storage costs.

• Statistics and data distribution influence plans.

Worked examples

A. Rank stores by revenue in each city

```
WITH store_sales AS (
  SELECT city, store_id, SUM(net_sales) AS revenue
  FROM daily_store_sales
  GROUP BY city, store_id
)
SELECT *,
```

```
       DENSE_RANK() OVER (
         PARTITION BY city ORDER BY revenue DESC
       ) AS city_rank
FROM store_sales;
```

B. Detect repeat purchases within 30 days

```
WITH ordered AS (
  SELECT customer_id, order_id, order_ts,
         LAG(order_ts) OVER (
           PARTITION BY customer_id ORDER BY order_ts, order_id
         ) AS previous_order_ts
  FROM orders
)
SELECT *,
       (order_ts - previous_order_ts) <= INTERVAL '30 days' AS within_30_days
FROM ordered
WHERE previous_order_ts IS NOT NULL;
```

C. Find gaps in a sequence of invoice numbers

```
WITH nums AS (
  SELECT invoice_no,
         LAG(invoice_no) OVER (ORDER BY invoice_no) AS prev_no
  FROM invoices
)
SELECT prev_no + 1 AS gap_start, invoice_no - 1 AS gap_end
FROM nums
WHERE prev_no IS NOT NULL
  AND invoice_no > prev_no + 1;
```

Guided exercises with solutions

1. Return the third most recent login per user.

```
WITH x AS (
 SELECT *, ROW_NUMBER() OVER (
   PARTITION BY user_id ORDER BY login_ts DESC, login_id DESC
 ) AS rn
 FROM logins
)
SELECT * FROM x WHERE rn = 3;
```

2. Find products whose revenue is above their category average.

```
WITH p AS (
 SELECT category, product_id, SUM(revenue) AS revenue
 FROM product_sales GROUP BY category, product_id
)
SELECT *
FROM (
 SELECT p.*,
        AVG(revenue) OVER (PARTITION BY category) AS category_avg
 FROM p
) x
WHERE revenue > category_avg;
```

3. Calculate cumulative daily active users from a daily summary table.

```
SELECT activity_date, active_users,
       SUM(active_users) OVER (
         ORDER BY activity_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_daily_user_days
FROM daily_activity;
```

This is cumulative user-days, not cumulative distinct users. A user can appear on multiple days.

4. Return users active on at least 5 distinct days in a 7-day period. Explain that this requires a per-user rolling date window, not simply a global daily count. One approach:

```
SELECT DISTINCT a.user_id
FROM activity a
WHERE (
  SELECT COUNT(DISTINCT b.activity_date)
  FROM activity b
  WHERE b.user_id = a.user_id
    AND b.activity_date BETWEEN a.activity_date - 6 AND a.activity_date
) >= 5;
```

5. Find each customer's longest gap between orders.

```
WITH x AS (
 SELECT customer_id, order_ts,
        LAG(order_ts) OVER (
          PARTITION BY customer_id ORDER BY order_ts
        ) AS prev_ts
 FROM orders
)
SELECT customer_id, MAX(order_ts - prev_ts) AS longest_gap
FROM x
WHERE prev_ts IS NOT NULL
GROUP BY customer_id;
```

Independent interview exercises

6. Return all tied employees at the highest salary in each department.

7. Find the first purchase date after signup.

8. Calculate 90th percentile order value by country.

9. Find users who had activity on three consecutive calendar days.

10. Explain why `COUNT(DISTINCT user_id)` across all events is not the same as summing daily active users.

Solutions

```
-- 6
SELECT *
FROM (
 SELECT e.*,
        RANK() OVER (
          PARTITION BY department_id ORDER BY salary DESC
        ) AS rnk
 FROM employees e
) x
WHERE rnk = 1;

-- 7
SELECT c.customer_id, MIN(o.order_ts) AS first_purchase
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'COMPLETED'
  AND o.order_ts >= c.signup_ts
GROUP BY c.customer_id;

-- 8 (PostgreSQL)
SELECT country,
       PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY order_value) AS p90
FROM orders
GROUP BY country;

-- 9 (PostgreSQL date arithmetic)
WITH d AS (
 SELECT DISTINCT user_id, activity_date
 FROM user_activity
), x AS (
 SELECT user_id, activity_date,
        LAG(activity_date, 1) OVER (PARTITION BY user_id ORDER BY activity_date) AS d1,
        LAG(activity_date, 2) OVER (PARTITION BY user_id ORDER BY activity_date) AS d2
 FROM d
)
SELECT DISTINCT user_id
FROM x
WHERE activity_date = d1 + 1
  AND d1 = d2 + 1;

-- 10
-- Daily active user counts overlap; the same person can be active on multiple days.
-- Sum daily DAU = user-days, not unique users over the entire period.
```

Mock test — advanced SQL (30 minutes, 30 marks)

• 8 marks: top two sellers per region including ties.

• 8 marks: latest valid status per shipment.

• 7 marks: week-over-week revenue with missing weeks.

• 7 marks: explain and correct an inflated aggregate after a one-to-many join.

Mark for correct grain, deterministic ordering, missing-period logic and clear explanation.

External practice for this module

1. LeetCode SQL 50 — complete the Advanced Select and Joins and Subqueries sections. Re-solve all Medium questions you miss.

2. LeetCode Advanced SQL 50 — use as optional extension if available to the account; do not pay solely for this module.

3. PostgreSQL Exercises — window functions — complete the window-function and date-related exercises.

4. DataLemur — solve 10 interview questions involving ranking, latest records, rolling metrics, dates or consecutive activity.

5. Use The Index, Luke — study index fundamentals and query performance; then inspect `EXPLAIN` plans in a local PostgreSQL database.

Exit requirement: solve 8 window/date problems in 60 minutes. Include latest-row-per-key, top-N-per-group, LAG/LEAD, rolling period and missing calendar periods. Explain why `ROWS` is not automatically equivalent to a calendar-day window.

Module 4 — DBMS, database design, transactions and performance

Learning outcomes

Explain how a relational database protects data, handles concurrency and executes queries; design a normalised schema; select indexes thoughtfully.

1. Normalisation and functional dependencies

A functional dependency A $\rightarrow$ B means A determines B. Normalisation reduces update, insertion and deletion anomalies.

• 1NF: attributes hold atomic values; no repeating groups.

• 2NF: 1NF and no non-key attribute depends on only part of a candidate key.

• 3NF: 2NF and no transitive non-key dependency on a key.

• BCNF: every determinant is a candidate key.

Example: `Enrollment(student_id, student_name, course_id, course_name, grade)`. Student name depends on student_id; course name depends on course_id; grade depends on the combination. Split into Students, Courses and Enrollments.

Denormalisation is a deliberate read-performance or analytical modelling choice, with consistency cost.

2. Transactions and ACID

A transaction groups operations into one logical unit.

• Atomicity: all or none.

• Consistency: constraints/invariants hold.

• Isolation: concurrent work obeys the configured isolation guarantee.

• Durability: committed work survives failures according to system guarantees.

```
BEGIN;
UPDATE wallets SET balance = balance - 250 WHERE wallet_id = 10;
UPDATE wallets SET balance = balance + 250 WHERE wallet_id = 20;
COMMIT;
```

If the second update fails, rollback the transaction. A production transfer should also validate sufficient balance and affected row counts.

3. Isolation anomalies

• Dirty read: read uncommitted data.

• Non-repeatable read: same row returns a different committed value on reread.

• Phantom: predicate query returns a changed set.

• Lost update: one concurrent write overwrites another.

Standard isolation labels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE. Exact guarantees differ by engine. PostgreSQL uses MVCC and does not implement READ UNCOMMITTED as a distinct weaker mode; do not assume identical behaviour across databases.

4. Locks and deadlocks

Shared/exclusive lock concepts protect reads/writes; actual lock modes differ by engine. A deadlock is a cycle of transactions waiting on resources held by each other. The database aborts a victim. Reduce risk with short transactions, consistent lock order, suitable indexes and safe retry.

5. Indexes

B-tree indexes commonly support equality and range lookups. Composite index order matters. Covering indexes may satisfy a query from index data alone. Indexes consume space and add write overhead. A low-selectivity index may not help much; use plans and workload evidence.

6. Query planning

An optimiser estimates cost and selects a plan. Understand sequential scan, index scan, nested-loop join, hash join, merge join, sort and aggregate. Compare estimated and actual rows. `EXPLAIN ANALYZE` executes the query; use caution on expensive or modifying statements.

7. Stored routines and database objects

• View: stored query definition.

• Materialized view: persisted query result requiring refresh/maintenance.

• Stored procedure: invoked server-side routine; may contain multiple statements and procedural logic.

• Function: returns scalar/table value and may be callable within expressions, subject to engine restrictions.

• Trigger: fires automatically on configured table events.

PostgreSQL procedure example:

```
CREATE OR REPLACE PROCEDURE mark_expired_subscriptions(p_cutoff date)
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE subscriptions
  SET status = 'EXPIRED'
  WHERE end_date < p_cutoff
    AND status = 'ACTIVE';
END;
$$;

CALL mark_expired_subscriptions(DATE '2026-01-01');
```

This syntax is PostgreSQL-specific. MySQL and SQL Server syntax and transaction restrictions differ.

8. DELETE, TRUNCATE and DROP

• DELETE removes selected rows; can use WHERE.

• TRUNCATE removes all rows efficiently in many engines; identity/transaction behaviour differs.

• DROP removes the object definition and data.

Know the target engine's transactional and permission semantics.

9. Partitioning, replication, sharding and backup

Partitioning divides a logical table into partitions; partition pruning can reduce scans. Replication keeps copies for availability/read scaling but may lag. Sharding distributes data across nodes and complicates cross-shard queries. Backup provides recoverability; replication alone is not a backup.

Guided lab

Create:

```
CREATE TABLE students (
  student_id BIGINT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT NOT NULL
);
CREATE TABLE courses (
  course_id BIGINT PRIMARY KEY,
  course_name TEXT NOT NULL
);
CREATE TABLE enrollments (
  student_id BIGINT REFERENCES students(student_id),
  course_id BIGINT REFERENCES courses(course_id),
  enrolled_on DATE NOT NULL,
  grade NUMERIC(5,2) CHECK (grade BETWEEN 0 AND 100),
  PRIMARY KEY (student_id, course_id)
);
```

Tasks:

1. Insert valid rows and demonstrate a foreign-key failure.

2. Try duplicate email and explain the constraint error.

3. Start a transaction, update a grade and rollback.

4. Add an index for course enrolment lookup.

5. Compare an EXPLAIN plan before and after, noting that a tiny table may still use a sequential scan.

6. Create a view showing course enrolment counts.

Lab answers / reasoning

• The composite primary key prevents a student enrolling in the same course twice.

• The foreign key prevents enrolment for a non-existent student/course.

• Rollback restores the previous committed state.

• An index is not guaranteed to be selected; table size and selectivity matter.

• A view stores the query definition, not necessarily the result.

Independent DBMS questions

1. Why is a unique constraint different from a primary key?

2. Why can a transaction be consistent but not isolated enough for a business rule?

3. What happens if an application retries a transfer after a timeout but the first commit actually succeeded?

4. Why can a trigger make debugging harder?

5. Why can a replica return stale data?

6. When is denormalisation justified?

Answer guide

1. A table may have multiple unique candidate keys but one primary key; NULL handling differs by engine.

2. Two individually valid operations can interleave and violate a cross-row invariant without suitable isolation/locking.

3. Use an idempotency key or unique transaction reference; do not blindly repeat a non-idempotent debit.

4. Side effects happen implicitly during DML and may be hidden from application flow.

5. Asynchronous replication lag.

6. Read-heavy analytical/reporting use cases where measured benefit outweighs update complexity.

External practice for this module

This module needs conceptual recall and hands-on database work; coding sites alone are not sufficient.

1. PostgreSQL documentation — constraints — read primary keys, unique, foreign keys and CHECK constraints; reproduce the lab in PostgreSQL.

2. PostgreSQL documentation — transactions and transaction isolation — compare isolation levels and reproduce a two-session concurrency example.

3. PostgreSQL documentation — indexes — create/drop B-tree indexes and compare plans.

4. Use The Index, Luke — complete the introductory chapters on indexes, composite indexes and execution plans.

5. GeeksforGeeks DBMS practice — use the MCQs for quick recall of keys, normalisation, transactions and concurrency; verify engine-specific claims against PostgreSQL documentation.

Hands-on assignment: create a small schema, demonstrate a constraint violation, commit and rollback, run two concurrent sessions, create an index and compare `EXPLAIN ANALYZE`. Keep screenshots or SQL scripts as evidence.

Exit requirement: answer 25 DBMS oral questions without notes and explain one anomaly, one deadlock scenario, one index trade-off and one idempotent transaction retry.

Module 5 — Python fundamentals and coding interview problem solving

Learning outcomes

Write small correct programs using core data structures, functions, loops, exceptions and tests. Explain time/space complexity.

1. Data structures and when to use them

• List: ordered, mutable sequence; index access O(1), membership O(n).

• Tuple: ordered immutable sequence.

• Set: unique values; average membership O(1).

• Dict: key-value mapping; average lookup/update O(1).

Choose based on required operations, not familiarity.

2. Mutability and identity

Mutable objects can change in place. == checks value equality; `is` checks object identity. Use `is None` for None checks. Avoid mutable default arguments.

3. Functions

Use descriptive names, small units of work, explicit return values and validated inputs. `*args` collects positional extras; `**kwargs` collects keyword extras.

4. Exceptions

Catch specific expected errors. Do not use bare `except` to hide bugs. Use context managers (`with open(...)`) for resources. Validate at system boundaries.

5. Comprehensions, sorting and standard library

Know `enumerate`, `zip`, `sorted(key=...)`, `collections.Counter`, `defaultdict`, `deque`, `heapq`, `datetime`, `csv`, `json`, `pathlib`.

6. Complexity basics

O(1), O(log n), O(n), O(n log n), O(n²). Explain both time and extra space. A nested loop is not automatically O(n²) if its bounds differ; analyse actual work.

Worked examples

Frequency count

```
from collections import Counter
def frequencies(values):
    return Counter(values)
```

Two-sum

```
def two_sum(nums, target):
    seen = {}
    for i, value in enumerate(nums):
        needed = target - value
        if needed in seen:
            return [seen[needed], i]
        seen[value] = i
    return None
```

Average O(n) time and O(n) space. Define whether returning one pair or all pairs is required.

Group records

```
from collections import defaultdict
def group_by_city(rows):
    grouped = defaultdict(list)
    for row in rows:
        grouped[row["city"]].append(row)
    return dict(grouped)
```

Guided exercises with solutions

1. Reverse words in a sentence while preserving word order reversal.

```
def reverse_words(sentence):
    return " ".join(sentence.split()[::-1])
```

2. Return common elements without duplicates.

```
def common_unique(a, b):
    return set(a) & set(b)
```

If output order matters, preserve the order from a:

```
def common_in_a_order(a, b):
    bset = set(b)
    seen = set()
    result = []
    for item in a:
        if item in bset and item not in seen:
            result.append(item)
            seen.add(item)
    return result
```

3. Find the most frequent item; on ties choose lexicographically smallest.

```
from collections import Counter
def most_frequent(values):
    if not values:
        return None
    counts = Counter(values)
    best_count = max(counts.values())
    return min(x for x, c in counts.items() if c == best_count)
```

4. Merge two sorted lists.

```
def merge_sorted(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

5. Validate balanced brackets.

```
def balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Independent exercises

6. Return the first repeated integer in scan order.

7. Rotate a list right by k positions.

8. Check whether two strings are anagrams, ignoring spaces and case.

9. Find the longest word in a sentence; define tie behaviour.

10. Implement a simple LRU cache conceptually; use `OrderedDict` for an implementation.

11. Find the maximum subarray sum (Kadane's algorithm).

12. Generate Fibonacci values lazily with a generator.

Solutions

```
# 6
def first_repeated(values):
    seen = set()
    for x in values:
        if x in seen:
            return x
        seen.add(x)
    return None

# 7
def rotate_right(values, k):
    if not values:
        return values
    k %= len(values)
    return values[-k:] + values[:-k] if k else values[:]

# 8
def are_anagrams(a, b):
    clean = lambda s: sorted(ch.lower() for ch in s if not ch.isspace())
    return clean(a) == clean(b)
```

```
# 9: first longest word wins
def longest_word(sentence):
    words = sentence.split()
    return max(words, key=len) if words else None

# 11
def max_subarray_sum(nums):
    if not nums:
        return None
    best = current = nums[0]
    for x in nums[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best

# 12
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b
```

For 10, the key design is a hash map plus doubly linked list (O(1) get/put); Python's `OrderedDict` offers a standard implementation.

Testing checklist

For every function test empty input, one element, duplicates, negative values, unexpected types, boundary k, ties and large input as appropriate.

Mini mock (35 minutes, 30 marks)

• 8 marks: implement a frequency-based function.

• 8 marks: solve a two-pointer problem.

• 6 marks: parse and validate JSON records.

• 4 marks: explain complexity.

• 4 marks: write at least four tests.

External practice for this module

1. HackerRank Python — complete Introduction, Basic Data Types, Strings, Sets, Collections, Functions and Exceptions. Aim for at least 20 problems.

2. Exercism Python track — complete 10 beginner exercises and read the mentor/community discussion after your own attempt.

3. LeetCode — Array and String problems — solve 8 Easy problems after learning lists, loops and functions.

4. LeetCode — Hash Table problems — solve 8 Easy problems after dictionaries and sets; then try 2 Medium problems.

5. NeetCode Roadmap — use the Arrays & Hashing and Two Pointers sections as a topic checklist, not as a race to finish every problem.

Exit requirement: solve 15 beginner/easy problems and 5 interview-style problems without copying. For every solution, state time and space complexity and test empty input, duplicates and boundary cases.

Module 6 — Python for data work: files, APIs, pandas and testing

Learning outcomes

Build a small, reliable data-processing script that reads files or APIs, validates records, transforms data, writes outputs and can be tested.

1. Files and formats

CSV is tabular text; JSON represents nested structured data; Parquet is a columnar analytical format. Choose formats based on consumers, size and schema needs. Use explicit encoding and date/number parsing.

```
import csv
with open("orders.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["order_id"])
```

2. APIs

Know HTTP methods, status codes, headers, authentication, pagination, rate limits, timeouts and JSON parsing. Use timeouts. Retry transient errors (429/5xx) with bounded exponential backoff and jitter; do not endlessly retry permanent 4xx errors.

3. pandas essentials

• Inspect: `head`, `sample`, `shape`, `info`, `describe`, `dtypes`.

• Missingness: `isna`, `notna`, `fillna`, `dropna`.

• Cleaning: `astype`, `to_numeric`, `to_datetime`, `drop_duplicates`.

• Transform: `assign`, vectorised operations, `map`, `groupby`, `agg`.

• Combine: `merge`, `concat`.

• Reshape: `pivot_table`, `melt`.

• Export: `to_csv`, `to_parquet`, `to_sql`.

4. Merge validation

```
joined = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one"
)
```

Use `validate` to catch unexpected key cardinality. Check unmatched keys after merge.

5. Missing values

Do not automatically fill every missing numeric value with zero. Ask whether missing means unknown, not applicable or true zero. Report missingness before treatment. For a date column, coercion may produce `NaT`; count and inspect those rows.

6. Testing and quality

Use unit tests for deterministic transformations; integration tests for database/API boundaries; data tests for uniqueness, not-null, accepted values, relationships, freshness and volume. Keep raw inputs and rejected records with reasons.

Guided exercises and solutions

Assume `orders.csv` has `order_id, customer_id, order_ts, status, amount`.

1. Parse dates and numeric amounts.

```
import pandas as pd
df = pd.read_csv("orders.csv")
df["order_ts"] = pd.to_datetime(df["order_ts"], errors="coerce", utc=True)
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
```

2. Report invalid required fields.

```
invalid = df[
    df["order_id"].isna()
```

```
    | df["customer_id"].isna()
    | df["order_ts"].isna()
    | df["amount"].isna()
]
```

3. Create monthly completed revenue.

```
valid = df[df["status"].eq("COMPLETED")].copy()
valid["month"] = valid["order_ts"].dt.to_period("M")
monthly = valid.groupby("month", as_index=False)["amount"].sum()
```

4. Detect duplicate order IDs and retain latest update.

```
df = df.sort_values(["updated_at", "ingested_at"])
deduped = df.drop_duplicates("order_id", keep="last")
```

This requires parsed timestamps and a defined tie rule.

5. Merge customer city, requiring one customer row per customer ID.

```
result = orders.merge(
    customers, on="customer_id", how="left", validate="many_to_one",
    indicator=True
)
unmatched = result[result["_merge"] == "left_only"]
```

Independent exercises

6. Calculate category revenue and item count from order lines.

7. Compare row count before and after a merge; explain unexpected growth.

8. Write a function that returns accepted and rejected rows with reason.

9. Read a 5 GB CSV in chunks and aggregate by month.

10. Write a unit test for a function that standardises status values.

11. Describe how to make a daily API ingestion job safe to rerun.

Solution patterns

• 6: `groupby("category").agg(revenue=("line_value","sum"), lines=("order_item_id","count"))`.

• 7: inspect key uniqueness, join cardinality, duplicate keys and null join keys; use pandas `validate`.

• 8: separate validation rules from persistence; preserve original row and error list.

• 9: `read_csv(..., chunksize=...)`, aggregate partial results, merge partial aggregates, checkpoint output.

• 10: test known mappings, unknown status, whitespace/case and null.

• 11: stable business key, deterministic transformation, upsert, run metadata and safe checkpoint.

Mini project

Create `daily_order_quality.py`:

1. Read CSV.

2. Validate required fields.

3. Parse timestamp and amount.

4. Reject invalid rows with reasons.

5. Deduplicate by order ID and latest update.

6. Produce daily revenue and quality summary.

7. Write output CSVs and run metadata.

8. Add tests for empty input, invalid amount, duplicate ID and missing timestamp.

External practice for this module

1. Kaggle Learn — Python — complete the Python micro-course if core syntax is still weak.

2. Kaggle Learn — Pandas — complete all lessons and exercises on indexing, grouping, missing data, merging and data types.

3. LeetCode — 30 Days of Pandas — complete the study plan; write down the pandas operation used and its SQL equivalent.

4. StrataScratch — select Python/Pandas interview problems involving groupby, joins, ranking and data cleaning.

5. pytest documentation — getting started — write tests for the mini-project in this module.

Exit requirement: complete 15 Pandas exercises and build one script that reads a file, validates, transforms, rejects bad rows with reasons, and writes reproducible outputs. Test empty input, duplicates and invalid types.

Module 7 — Statistics and product analytics

Learning outcomes

Define product metrics, interpret distributions, reason about uncertainty and design a basic A/B test.

1. Descriptive statistics

Mean is sensitive to extreme values; median is robust to skew. Standard deviation measures spread around the mean. Percentiles describe position in a distribution. IQR = Q3 − Q1 and is useful for robust outlier review.

Exercise

Delivery times are 12, 14, 15, 16, 93 minutes.

• Mean = 30 minutes.

• Median = 15 minutes.

The mean is pulled up by the 93-minute delay. Report median and p90/p95 as well as mean where tail experience matters.

2. Probability and sampling

Understand population, sample, sampling bias, selection bias, survivorship bias, independent events and conditional probability. A large biased sample can still be misleading.

3. Confidence intervals and p-values

A confidence interval describes uncertainty under a statistical procedure. A p-value is the probability, assuming the null model, of observing data at least as extreme as what was observed. It is not the probability that the null hypothesis is true. Statistical significance does not guarantee meaningful business impact.

4. A/B testing

• Define hypothesis and primary metric.

• Select randomisation unit (user, account, session).

• Define eligibility and exclusion rules.

• Choose minimum detectable effect, alpha and power; estimate sample size.

• Monitor sample ratio, instrumentation and guardrails.

• Avoid unplanned peeking unless using a valid sequential design.

• Report effect size and confidence interval, not just p-value.

5. Product metrics

Conversion = converted eligible entities / eligible entities.

Retention requires cohort, return event and time window.

Churn requires a clear cancellation/inactivity rule.

DAU/WAU/MAU require a definition of active and timezone.

ARPU = revenue divided by a stated user base.

Funnel conversion must define event ordering and session/user grain.

Guided case 1 — Orders are down

A marketplace sees completed orders fall 9% week over week.

Steps:

1. Validate data freshness, tracking and status definitions.

2. Compare traffic, conversion and orders per buyer.

3. Segment by channel, geography, device and new/returning users.

4. Check seasonality, campaigns, release and payment outages.

5. Quantify segment contribution before recommending action.

Guided case 2 — Feature adoption

A new wishlist feature has 35% adoption among users who saw the prompt.

Ask whether exposure is random, whether users who saw the prompt differ, and whether adoption improves retention or purchase. Adoption alone is not proof of value.

Independent exercises with answer guidance

1. AOV rises while revenue falls. Explain possible drivers: fewer orders, mix shift, fewer buyers, higher basket value.

2. DAU rises but 7-day retention falls. Segment acquisition cohorts and acquisition channel; check whether low-intent users were acquired.

3. A promotion increases conversion but reduces margin. Calculate incremental contribution, not just conversion lift.

4. An A/B test shows +1.2% conversion with a wide confidence interval crossing zero. State uncertainty; do not claim a proven lift.

5. A dashboard and finance disagree on revenue. Reconcile event status, refunds, timezone, currency, tax, duplicates and recognition rules.

Product case answer structure

Clarify metric → validate data → decompose → segment → quantify → identify evidence → recommend next action → state caveats.

Mini assessment (30 minutes, 30 marks)

• Define a retention metric precisely (5).

• Interpret skewed delivery-time data (5).

• Design an A/B test for checkout (10).

• Diagnose a revenue/finance discrepancy (10).

External practice for this module

1. Khan Academy — Statistics and Probability — practise distributions, mean/median, sampling, probability and confidence intervals.

2. Seeing Theory — use the interactive visual explanations for probability, distributions and inference.

3. StatQuest — watch the relevant lessons on p-values, confidence intervals, hypothesis tests and statistical power; after each, write a plain-English explanation without replaying the video.

4. Evan Miller — A/B testing tools and articles — study sample size and experiment design; understand assumptions before using a calculator.

5. DataLemur — practise business metric SQL questions involving conversion, retention, cohorts and revenue.

Applied assignment: choose a public dataset from Kaggle Datasets, define 3 metrics with explicit denominators, analyse a segment, explain one confounder and write a one-page recommendation. Do not claim causal impact from observational data.

Exit requirement: explain mean vs median, sampling bias, confidence interval, p-value, statistical vs practical significance and an A/B test design using a product example.

Module 8 — Excel, Power Query, Power BI and dashboard interviews

Learning outcomes

Prepare data, calculate metrics, model a star schema, write basic DAX and build dashboards that support decisions.

1. Excel

Know relative/absolute references, tables, sort/filter, text/date functions, `IF`, `IFERROR`, `SUMIFS`, `COUNTIFS`, `XLOOKUP`, PivotTables, charts, slicers and conditional formatting.

Exercise

Given `Sales(Date, Region, Product, Units, NetSales)`, calculate revenue by region and product, identify the top 5 products, and show monthly trend using a PivotTable. Reconcile PivotTable total to source sum.

2. Power Query

Import → profile → change types → clean → merge/append → load. Merge combines columns using a join; append stacks rows. Keep transformations repeatable and inspect errors.

3. Data model

A star schema contains a fact table at a declared grain and dimensions describing it. Example:

• FactSales: one row per order line.

• DimDate: one row per date.

• DimProduct: one row per product.

• DimStore: one row per store.

Use one-to-many relationships from dimensions to fact where appropriate.

4. Measures vs calculated columns

Measures calculate in filter context at query time. Calculated columns are computed during refresh and stored. Use measures for aggregations that should respond to report filters.

5. DAX examples

```
Net Sales = SUM(FactSales[NetSales])

Units Sold = SUM(FactSales[Units])

Order Count = DISTINCTCOUNT(FactSales[OrderID])

Average Order Value =
DIVIDE([Net Sales], [Order Count])
```

If FactSales is order-line grain, distinct order count is needed. AOV definition should use the agreed revenue and order population.

6. Dashboard design

Audience and decision first. Use clear metric definitions, units, date range, refresh timestamp, trends and useful segmentation. Avoid chart clutter, truncated axes, unlabelled denominators and colour-only meaning.

Independent exercises

1. Explain why a date dimension is needed for time intelligence.

2. A measure doubles after adding Product dimension. Investigate relationships and duplicate keys.

3. Design a dashboard for fulfilment operations with five KPIs and a drill-down.

4. Explain Import vs DirectQuery trade-offs.

5. Describe row-level security and how to test it.

6. Explain how you would reconcile a Power BI KPI with SQL.

Answer guidance

1. Date dimension provides continuous calendar attributes and supports time calculations.

2. Check relationship cardinality, dimension uniqueness, bridge tables and filter direction.

3. Include on-time delivery, backlog, average/p90 fulfilment time, cancellation and volume; define denominator/time window.

4. Import often gives performance and modelling flexibility; DirectQuery queries the source and depends on source capacity/latency and feature limitations.

5. Restrict rows by user/role; test with representative identities and least privilege.

6. Match grain, filters, timezone, status, currency, refresh time and NULL handling.

External practice for this module

1. Microsoft Learn — Power BI — complete beginner learning paths on getting data, modelling and building reports.

2. Microsoft Learn — DAX — use the reference for SUM, CALCULATE, filter context and time intelligence.

3. SQLBI — study introductory articles/videos on star schema, relationships, row context and filter context.

4. DAX Guide — look up functions while completing the measures in this module.

5. ExcelJet — practise SUMIFS, COUNTIFS, XLOOKUP, date/text formulas and PivotTables.

Applied assignment: download a dataset from Maven Analytics Data Playground or Kaggle. Build a star schema, 5 measures, 1 time trend, 2 useful slicers and a dashboard page. Reconcile every KPI against SQL or a spreadsheet total.

Exit requirement: explain fact grain, dimension relationships, measure vs calculated column, filter context and how a dashboard supports a specific decision.

Module 9 — Data modelling, warehouses and ETL

Learning outcomes

Choose table grain, model facts and dimensions, understand history handling, and design a robust batch pipeline.

1. OLTP vs OLAP

OLTP supports operational transactions and current state. OLAP supports analytical scans, aggregations and historical analysis. Their schema and performance goals differ.

2. Fact and dimension tables

Fact table records measurable business events or snapshots. Dimension table describes entities such as product, customer, store or date. Declare grain before selecting columns.

3. Star and snowflake schemas

Star schema is easier for many BI queries; snowflake normalises dimension hierarchies but can require more joins. Choose based on usability and governance.

4. Slowly changing dimensions

• Type 1: overwrite old value; no history.

• Type 2: create a new version with effective start/end dates and current flag.

Use Type 2 when historical reporting must reflect attributes as they were at the event time.

5. ETL and ELT

ETL transforms before loading; ELT loads raw data then transforms in the analytical platform. Choose based on data sensitivity, latency, platform capability and cost.

6. Incremental loading

Watermark tracks last processed time/sequence. Use overlap/lookback for late updates. CDC captures inserts/updates/deletes from logs or change streams. MERGE/upsert applies changes by stable key. Backfills reprocess historical ranges.

7. Idempotency and exactly-once

A rerun should not create duplicate or inconsistent output. Stable keys, deterministic transformations and upserts make processing idempotent. Exactly-once end-to-end is difficult; at-least-once delivery plus idempotent processing is common.

8. Reliability and data quality

Monitor freshness, volume, schema, null rate, uniqueness, distribution, duration and failures. Quarantine invalid rows with reasons. Preserve raw inputs for replay subject to retention/privacy rules.

Worked design

Daily order API → raw immutable files → staging validation → dedup latest order version → curated fact/dimensions → quality/reconciliation checks → BI.

Track `run_id`, extraction time, source cursor, row counts, rejected counts and output partition.

Exercises with answer guidance

1. Model order header and order lines. Explain why one fact table at order-line grain differs from order-grain.

2. Customer changes city; historical reports need old city. Use Type 2.

3. Source sends late order update from 12 days ago. Reprocess overlapping window and MERGE by order ID/version.

4. Job fails after writing staging but before publish. Use run state/atomic publish or partition swap; do not expose incomplete output.

5. Design a reconciliation: source count/value vs target count/value by date/status; explain expected exclusions.

6. Explain why a backup is needed even with replication.

7. Explain how to handle a new optional source column vs a missing required column.

Mini design test (35 minutes, 35 marks)

Design a daily inventory snapshot model:

• declare grain;

• define dimensions and measures;

• explain late-arriving corrections;

• choose partition strategy;

• define quality checks;

• describe backfill and dashboard freshness.

External practice for this module

1. dbt Learn — complete introductory material on analytics engineering, models, tests and documentation.

2. DataTalksClub Data Engineering Zoomcamp — use the data warehouse, data ingestion and orchestration modules as a guided lab; select the relevant sections rather than trying to finish the whole programme immediately.

3. Kimball Group — dimensional modelling techniques — read about fact tables, dimensions, grain and slowly changing dimensions.

4. PostgreSQL documentation — INSERT/ON CONFLICT — practise upsert patterns for idempotent loads.

5. NYC Open Data — select a dataset and design a raw → staging → curated pipeline with quality checks.

Applied assignment: model a retail order-line fact table, customer/product/date dimensions, and a Type 2 customer dimension. Load data twice and prove the second run does not duplicate output.

Exit requirement: draw a pipeline, state table grain, describe late-arriving updates, backfills, deduplication, data quality checks and safe reruns.

Module 10 — Data engineering systems, cloud and Spark foundations

Learning outcomes

Explain batch vs streaming, orchestration, distributed processing and basic cloud architecture without overstating hands-on experience.

1. Batch and streaming

Batch processes bounded data on a schedule. Streaming processes events continuously or near-real-time. Streaming adds event time vs processing time, late events, ordering, checkpointing and replay complexity.

2. Orchestration

A DAG describes task dependencies. Orchestrators schedule, retry, monitor and backfill tasks; they do not necessarily perform the heavy data transformation. Know task dependency, retry, timeout, sensor, schedule and idempotency.

3. Cloud building blocks

• Object storage for raw files and data lake zones.

• Managed databases/warehouses for query and serving.

• Compute services for transformation.

• Identity/access management and secret storage.

• Logging, metrics, alerting and network controls.

Learn concepts first; AWS, Azure and GCP product names vary.

4. Spark fundamentals

Spark DataFrames are distributed tabular abstractions. Transformations are lazy; actions trigger execution. Shuffles move data across partitions and are often expensive. Skew means some partitions contain much more data than others. Broadcast joins can help when one side is small enough. Cache only when reused and measured.

5. File and table formats

CSV is simple but weakly typed and verbose. Parquet is columnar and compressed, suited to analytical scans. Delta Lake, Iceberg and Hudi add table metadata/transaction and evolution capabilities over data files; details differ.

6. Partitioning

Partition on columns used for common pruning and with sensible cardinality. Too many tiny partitions/files increase metadata and scheduling overhead. Partitioning is not the same as indexing or sharding.

7. Security and governance

Least privilege, secret management, encryption, PII minimisation, retention, audit logs, lineage and data ownership. Never put credentials in Git or logs.

Exercises with answer guidance

1. Design a 15-minute latency event pipeline. Discuss broker, consumer, checkpoint, event-time windows and late events.

2. A Spark job is slow after a join. Inspect shuffle, skew, partition counts, join strategy and file sizes.

3. Why can 50,000 tiny Parquet files be worse than fewer larger files? Metadata and task overhead.

4. What happens if an orchestrated task retries after writing output? Make output idempotent or use atomic publish.

5. Explain how to backfill 3 months without duplicating current data.

6. What metrics would alert you before a 7 AM freshness SLA is missed?

Interview boundary

If you studied Spark but did not use it in a project, say “I understand the fundamentals and have practised…” rather than claiming production Spark experience.

External practice for this module

1. DataTalksClub Data Engineering Zoomcamp — work through selected Docker, cloud, orchestration, Spark and warehouse labs.
2. Apache Spark — Quick Start — run DataFrame transformations and inspect lazy evaluation.
3. Apache Airflow — Tutorials — create a small DAG with dependencies, retries and task logging.
4. AWS Skill Builder — learn cloud fundamentals, IAM, S3 and managed data services; use free learning content where available.
5. Databricks Academy — use introductory lakehouse and Spark learning resources.

Applied assignment: build or simulate a daily batch pipeline with raw files, validation, transformation, a warehouse table, run metadata and a rerun test. Then explain how the architecture changes for streaming.

Exit requirement: explain batch vs streaming, event time, checkpointing, shuffle, skew, partition sizing, object storage, IAM and recovery after partial failure. Do not claim production experience from completing tutorials.

Module 11 — Project defence and resume interview lab

This module is specific to the three portfolio projects in the preparation plan. Confirm every detail against the actual repository before using it in an interview.

Project walkthrough template

1. User/problem: who needs this and why?

2. Source: where does data come from, and what is the licence/refresh constraint?

3. Grain: what does one source and output row represent?

4. Architecture: ingestion → validation → transform → storage → analysis/UI.

5. Design decisions: why these tools?

6. Quality: tests, validation, reconciliation and rejected records.

7. Failure: one real issue and how it was debugged.

8. Limitations: what is not implemented or not validated?

9. Scale: what changes at 10x volume?

10. Next step: one measurable improvement.

RetailPulse

Explain the retail sales/inventory dataset, source files, transformation pipeline, SQLite warehouse and Streamlit/Plotly dashboard. Be careful: profitability measures should not be presented as validated until the source cost-field mapping is verified. Do not infer transactions from row count if order/transaction grain is not established. Do not claim unsupported customer/returns/geography features.

RetailPulse exercises

• Trace one raw row into a curated table and dashboard metric.

• Explain how you validate key uniqueness and missing values.

• Explain why raw source files are not committed if size limits apply.

• Design a test for category revenue reconciliation.

• Propose a safe incremental refresh if the source publishes new data.

WeatherScope India

Explain daily forecast ingestion from Open-Meteo for five cities, transformation, quality checks, SQLite, reports and Power BI/Streamlit. Row grain is city × forecast date × fetch timestamp. Forecasts are predictions, not observed weather or official warnings. A 50% precipitation probability threshold is a project-defined rule, not an official alert. Do not claim scheduler or production alerting unless implemented.

WeatherScope exercises

• Why is fetch timestamp part of the grain?

• How would you select the latest forecast per city/date?

• How would you compare forecast to actual observed weather if a reliable observation source were added?

• What happens if one city API call fails?

• How do you prevent mixing forecasts from different fetch runs?

TrustLayer

Explain NYC 311 data quality validation, accepted/rejected records, SQLite, pipeline metrics, CSV/JSON summaries, pytest and GitHub Actions. Synthetic test data is for deterministic testing and is not real NYC statistics. A single 1,000-row extract with zero rejects does not prove the entire source is clean. No live dashboard or scheduler is claimed.

TrustLayer exercises

• Design a new validation rule for impossible date chronology.

• Explain why rejected records should retain reason and source row.

• Add a test for duplicate request IDs.

• Explain how API pagination and rate limits would be handled.

• What is the difference between row rejected and pipeline run failed?

Resume defence

For every bullet, prepare:

• what you personally did;

• tools used and why;

• exact input/output;

• evidence (code, tests, screenshot, result);

• limitation;

• what you would improve.

Do not invent business impact, production scale, user count or performance improvement. If you cannot reproduce a metric, remove it or qualify it.

STAR stories

Prepare truthful examples for learning, debugging, ambiguity, feedback, collaboration and a mistake. Situation and task should be brief; actions should focus on your own decisions; result should be concrete but not exaggerated.

External practice and evidence links

Use these to improve the engineering quality of the existing portfolio projects—not to replace the project facts documented above.

1. GitHub Skills — complete Introduction to GitHub and GitHub Actions; apply version control and CI concepts to the repositories.
2. GitHub Actions documentation — add or review automated tests and a clear workflow status.
3. Streamlit Get Started — review app structure, caching and deployment practices for RetailPulse and WeatherScope.
4. Open-Meteo API documentation — verify request parameters, response fields and forecast semantics used by WeatherScope.
5. NYC Open Data — explore the source and data dictionary before proposing new TrustLayer validation rules.
6. Kaggle Datasets — find a separate small dataset for a new extension; do not silently replace the documented source of an existing project.

Evidence task: for each project, add a reproducible setup section, architecture diagram, sample input/output, tests, known limitations and one screenshot captured from the actual running application. Never invent performance metrics or screenshots.

Module 12 — Role-specific fresher mock interviews

These mocks are intentionally different from the concept exercises. They test whether you can combine skills under time pressure.

Mock A — Data Analyst / Product Analyst (90 minutes, 100 marks)

1. SQL analysis — 25 marks, 25 min: identify customers whose second purchase occurred within 60 days; calculate repeat rate by signup cohort.

2. Excel/BI — 15 marks, 15 min: propose a report for subscription renewals and explain the model.

3. Statistics — 15 marks, 15 min: interpret a test result with effect size and confidence interval.

4. Product case — 25 marks, 25 min: free-trial activation falls 8 percentage points.

5. Communication — 20 marks, 10 min: present finding, evidence, recommendation and caveat.

Answer expectations

• SQL must define cohort, completed purchase and date window.

• BI must define fact grain, dimensions, measures and reconciliation.

• Statistics must not confuse p-value with probability the null is true.

• Product case must validate instrumentation, decompose funnel and segment.

• Communication must distinguish fact from hypothesis.

Mock B — Junior Data Engineer (100 minutes, 100 marks)

1. SQL — 25 marks, 25 min: latest inventory state per SKU/store; detect source duplicates.

2. Python — 20 marks, 20 min: parse nested JSON, validate required fields, quarantine invalid records.

3. DBMS — 15 marks, 15 min: transaction, index and isolation scenario.

4. Pipeline design — 25 marks, 25 min: daily supplier feed with corrections and missing files.

5. Reliability/behavioural — 15 marks, 15 min: explain a failed run, retry and communication.

Answer expectations

• Deterministic latest-row selection.

• Python separates parsing, validation and persistence.

• DBMS answer includes engine-specific caveat.

• Design includes raw retention, idempotency, late corrections, backfill, alerting and SLA.

• Behavioural response states actual contribution.

Mock C — Mixed entry-level data role (60 minutes, 60 marks)

• 15 min SQL: rank sellers by region.

• 10 min Python: frequency and deduplication.

• 10 min DBMS: explain ACID and index trade-offs.

• 15 min business case: revenue flat but active buyers down.

• 10 min project defence: architecture and one limitation.

Marking guide

Award full marks for correct result plus assumptions, edge cases and explanation. Award partial marks for a correct approach with a small syntax issue if the candidate can debug it. Deduct for unsupported assumptions, silent data loss, incorrect metric denominator or fabricated project claims.

Interview communication framework

For coding: clarify → state approach → code → test → complexity.

For analytics: define metric → validate data → decompose → quantify → recommend.

For design: requirements → architecture → data model → failure handling → monitoring/security → trade-offs.

For behavioural: STAR, with specific personal actions and honest outcomes.

External timed practice for this module

Analyst / Product Analyst track

• DataLemur: complete 10 mixed SQL questions tagged easy/medium, prioritising joins, aggregation, ranking and product metrics.

• StrataScratch: complete 5 business-oriented SQL questions; explain assumptions before coding.

• LeetCode SQL 50: complete the full plan, then reattempt failed questions under a 45-minute limit.

• Kaggle Learn: complete Python and Pandas exercises; perform one independent dataset analysis.

Junior Data Engineer track

• HackerRank SQL: complete 10 mixed SQL problems.

• LeetCode: solve 10 Easy and 3 Medium array/hash-map problems.

• Exercism Python: complete 5 exercises and review edge cases.

• DataTalksClub Zoomcamp: complete one selected ingestion or warehouse lab and explain the architecture.

Final rule

Do not count a problem as completed merely because the platform accepted the code. Re-solve it from a blank editor after 3–7 days and explain complexity, grain, assumptions and edge cases aloud.

Python concept checks and final test

Check 1 — data structures (10 marks)

1. Choose a structure for fast membership checks over unique IDs.

2. Preserve insertion order while removing duplicates.

3. Explain mutable default argument risk.

Answers: set; loop with a seen set and output list; default list is reused across calls.

Check 2 — functions/errors (10 marks)

Write a function to parse a date string and return `None` for invalid input. Explain why catching broad Exception is dangerous.

Answer: use `datetime.strptime` in a try/except `ValueError`; catch expected failures only.

Final Python test (45 minutes, 40 marks)

Input is a list of web click events: `visitor_id`, `page`, `timestamp`, `campaign`.

1. Validate required fields and timestamp (10).

2. Remove duplicate event IDs if present; define tie rule (8).

3. Compute page views by campaign (8).

4. Find visitors who viewed pricing and later viewed checkout (8).

5. Write tests for empty, invalid, duplicate and out-of-order data (6).

Solution approach

Use a validation function returning `(accepted, rejected)`; parse timestamps to aware datetime; sort per visitor by timestamp; aggregate with dictionaries or pandas. “Later” means strictly later timestamp; define same-timestamp tie handling. Preserve rejected records and reasons.

SQL concept checks and final test

Check 1 — SELECT/filter/NULL (10 marks)

1. Find rows where `delivered_at` is missing.

2. Find orders in 2025 with amount >= 1000.

3. Explain `COUNT(*)` vs `COUNT(delivered_at)`.

Answers: `IS NULL`; half-open 2025 range and amount predicate; first counts all rows, second counts non-NULL timestamps.

Check 2 — joins/aggregation (15 marks)

Given invoices and invoice_lines, calculate invoice count and line revenue by month without multiplying invoice count.

Answer: aggregate invoices at invoice grain separately from line revenue or count distinct invoice IDs after joining; define status/date rules.

Check 3 — windows (15 marks)

Return each account's first failed payment and the next successful payment after it.

Answer approach: rank/filter failed payments; join to later successful payments; use `MIN(success_ts)` per account or a lateral query; clarify whether “next” is per account or per failure event.

Final SQL test (45 minutes, 40 marks)

A food delivery company has `restaurants`, `deliveries`, `delivery_events`, `customers`.

1. Compute completed delivery count and median delivery duration by city (10).

2. Find customers with 3+ completed deliveries in 30 days (8).

3. Return latest event per delivery (8).

4. Identify restaurants whose cancellation rate exceeds city average (8).

5. Explain a duplicate join issue and how you validate grain (6).

Solution notes

• Duration: completed timestamp minus accepted timestamp; exclude/flag negative or missing durations.

• 30-day condition: define rolling 30 days vs calendar month.

• Latest event: `ROW_NUMBER` with event timestamp and event ID tie-break.

• Cancellation rate: cancelled / eligible delivery attempts; guard denominator with `NULLIF`.

• City average: clarify weighted average of all deliveries vs average of restaurant-level rates.